

DDD Avanzado & Arquitectura Hexagonal

De la Teoría Táctica a la Implementación Limpia en C# .NET10

ARQUITECTURA ENTERPRISE

C# 14 / .NET 10

DDD TÁCTICO & ESTRATÉGICO

Un recorrido práctico sobre diseño de software robusto aplicado a Bounded Contexts de
Beneficios, Préstamos, Ahorros, Operaciones, Afiliación y Contabilidad.

Iniciar Presentación →

Recapitulación: Pilares Fundamentales de DDD

El vocabulario y conceptos base que debemos dominar antes de avanzar

DIMENSIÓN ESTRATÉGICA

- **Ubiquitous Language:** Lenguaje común y riguroso compartido entre expertos de negocio y desarrolladores sin modismos técnicos.
- **Bounded Context:** Frontera explícita en el modelo dentro de la cual los términos técnicos y reglas de negocio tienen significado unívoco.
- **Subdominios:** Clasificación del problema de negocio en *Core Subdomain* (ventaja competitiva), *Supporting* y *Generic*.

Focus: Delimitación de fronteras y alineación con la organización.

DIMENSIÓN TÁCTICA

- **Entities & Value Objects:** Entidades con identidad e historia de vida vs Objetos de Valor inmutables definidos 100% por sus atributos.
- **Aggregate & Aggregate Root:** Clúster de objetos del dominio agrupados bajo una raíz (Root) para garantizar consistencia e invariantes en una transacción.
- **Domain Events:** Notificación explícita de un hecho relevante del negocio que ya ocurrió, utilizado para desacoplar efectos secundarios.

Focus: Modelado rico en C# respetando invariantes y encapsulación.

I. El Puente Arquitectónico

Del Conocimiento Básico al DDD Enterprise

⚠ EL PROBLEMA HABITUAL

- **Anemia en el Dominio:** Entidades reducidas a meros DTOs con getters/setters públicos indeseados.
- **Lógica dispersa:** Validaciones críticas filtradas en controladores HTTP o Application Services.
- **Acoplamiento a Infraestructura:** Dependencias directas de EF Core o SDKs de mensajería en el modelo.
- **Dual-Write Frágil:** Intentar guardar en BD y publicar en RabbitMQ en la misma petición web.

🛡 EL ENFOQUE SENIOR DE HOY

- **Screaming Architecture:** El código refleja el negocio, no el framework.
- **Invariantes Incondicionales:** Constructores privados, Factory Methods y *Design by Contract*.
- **Fronteras con ACL:** Aislamiento defensivo entre Bounded Contexts vía *Anti-Corruption Layer*.
- **Outbox Pattern:** Persistencia atómica de eventos de dominio con interceptores de EF Core.

2. Screaming Architecture & Subdominios

Delimitación Estratégica del Sistema de Beneficios

CORE DOMAINS

NÚCLEO

Lógica diferenciadora del negocio

-  **Productos BC:** Políticas, tasas, accesorios y comisiones.
-  **Préstamos BC:** Amortización, intereses moratorios y devengados.
-  **Ahorros BC:** Captación, cuentas, prelación de retiros y rendimientos.
-  **Operaciones BC:** Orquestación de refinanciamientos y ampliaciones.



SUPPORTING

SOPORTE

Capacidades de apoyo especializado

-  **Afiliación BC:** Ciclo de vida y estado de afiliados.
-  **Contabilidad BC:** Enteros bancarios y dispersión PAG.



GENERIC

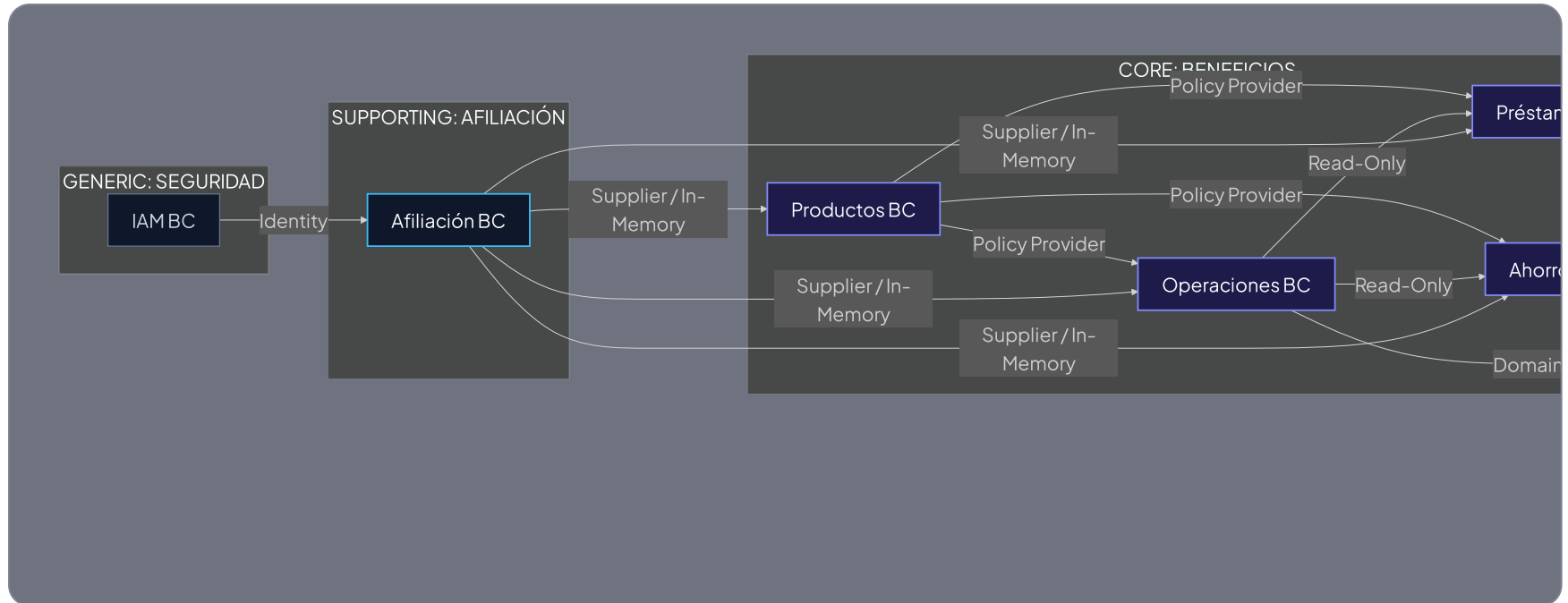
INFRA

Capacidad transversal reutilizable

-  **IAM BC:** Autenticación, tokens JWT, roles y auditoría de accesos al sistema.

3. Context Map & Relaciones Upstream / Downstream

Visión General del Sistema de Beneficios



4. Estrategias de Integración entre Contextos

Relaciones entre Equipos y Modelos de Dominio

Clasificación Upstream / Downstream

Upstream (U) / Downstream (D):

El contexto Upstream dicta el contrato o emite eventos; el Downstream consume y se adapta.

Customer-Supplier:

El equipo Upstream atiende las necesidades de integración del Downstream con prioridad.

Published Language (PL):

Contratos DTO inmutables y compartidos (ej. eventos en

`SharedKernel`).

Patrones Tácticos de Integración

Anti-Corruption Layer (ACL):

Traductor defensivo que evita que modelos o DTOs externos contaminen el Core Domain.

Conformist Pattern:

El Downstream acepta el esquema Upstream cuando este es estable y estándar.

Shared Kernel:

Código compartido strictly acotado a Value Objects agnósticos

(ej. `Dinero.cs`).

5. Anti-Corruption Layer (ACL) en C# 14

Aislando el Dominio de Préstamos del Contexto de Productos

El contexto de Préstamos requiere reglas del contexto de Productos, pero NUNCA debe acoplarse a los DTOs externos.

```
// 1. Contrato de la Capa Anticorrupción (ACL) en Persistence/Mappers/IMapper.cs
public interface IMapper<TDomain, TPersistence>
{
    TDomain MapToDomain(TPersistence persistence);
    TPersistence MapToPersistence(TDomain domain);
    void MapToExistingPersistence(TDomain domain, TPersistence persistence);
}

// 2. Mapeador ACL Concreto: Traduce de PrestamoDbEntity (BD) a Prestamo (Dominio)
public class PrestamoMapper : IMapper<Prestamo, PrestamoDbEntity>
{
    public Prestamo MapToDomain(PrestamoDbEntity persistence)
    {
        var id = new PrestamoId(persistence.Id);
        var beneficiarioId = new BeneficiarioId(persistence.BeneficiarioId);
        var origen = new OrigenApertura(persistence.OrigenConcepto, persistence.OrigenFolio);
        var saldoPendiente = new Dinero(persistence.SaldoPendiente, persistence.Moneda);
        var estado = MapearEstadoDesdeCodigo(persistence.EstadoCodigo);
```

6. Domain Services vs Application Services

Responsabilidad Exacta en la Arquitectura

Domain Service

Lógica de negocio pura multientidad.

-  Sin I/O ni dependencias de BD.
-  Sin control transaccional.
-  Algoritmos y cálculos puros.

Application Service

Orquestador del Caso de Uso.

-  Carga Agregados vía Repositorio.
-  Invoca Domain Services.
-  Persiste vía UnitOfWork.

Infrastructure Service

Detalles de entrada / salida y HTTP.

-  Conexión con APIs bancarias (PAG).
-  Generación de PDFs y Mails.
-  Interceptores de EF Core.

```
// Domain Service Puro: Lógica de negocio multientidad sin I/O ni repositorios
public class CalcularSaldoService
{
    public static ResultadoCalculo ValidarYCalcularPago(
        Prestamo prestamo, EstadoCuenta estadoCuenta, Dinero monto, Periodo periodo)
    {
        // Validar reglas de dominio entre Agregados (Préstamo y EstadoCuenta) y retornar cálculo
        return new ResultadoCalculo(monto, periodo);
    }
}
```


7. Strongly Typed IDs & Performance en C# 14

Optimización de Memoria: readonly record struct vs record

Garantizamos seguridad de tipos sin saturar el Garbage Collector(GC):

1 Campo Primario -> struct

ZERO HEAP
ALLOCATION

Se asigna en el Stack. Cero presión en el GC en consultas masivas.

```
public readonly record struct PrestamoId(Guid Value)
{
    public static PrestamoId New() => new(Guid.NewGuid());
    public static PrestamoId Empty => new(Guid.Empty);
}
```

Compuestos / Grandes -> record

PASS BY
REFERENCE

Para llaves compuestas con metadatos. Evita copias pesadas en Stack.

```
public record CompositeKeyId(
    string TenantCode,
    long SequenceId
);
```

8. Value Objects Ricos & Fowler Allocation Algorithm

Dominio Financiero sin Pérdida de Centavos en Dinero.cs

Los Value Objects inmutables encapsulan reglas de cálculo exacto:

```
public record Dinero
{
    public decimal Monto {; init; }
    public string Moneda {; init; }

    private Dinero(decimal monto, string moneda) =>
        (Monto, Moneda) = (monto >= 0 ? monto : throw new DomainException("Monto negativo"), moneda);

    public static Dinero Decimal(decimal monto, string moneda = "MXN") => new(monto, moneda);

    // Fowler Allocation Algorithm: Distribución equitativa prorrateada sin perder centavos
    public Dinero[] DistribuirProporcionalmente(int partes)
    {
        var montoBase = Math.Truncate((Monto / partes) * 100m) / 100m;
        var residuoCentavos = (int)((Monto - (montoBase * partes)) * 100m);

        var resultado = new Dinero[partes];
        for (int i = 0; i < partes; i++)
        {
            var extra = i < residuoCentavos ? 0.01m : 0.00m;
```

9. Factory Methods & Reconstitución de Dominio

Constructor Privado Único: NuevoPrestamo vs Reconstruir

El Agregado protege su encapsulación prohibiendo constructores públicos tradicionales:

```
public class Prestamo : AggregateRoot<PrestamoId>
{
    // 1. CONSTRUCTOR PRIVADO ÚNICO: Único canal de instanciación controlado
    private Prestamo(PrestamoId id, BeneficiarioId beneficiarioId, OrigenApertura origen,
        CondicionesOriginales condiciones, Dinero saldoInicial, EstadoPrestamo estado) : base(id) { ... }

    // 2. FACTORY METHOD DE NEGOCIO (Creación Nueva desde Caso de Uso en Aplicación)
    public static Prestamo NuevoPrestamo(PrestamoId id, BeneficiarioId beneficiarioId,
        CondicionesOriginales condiciones, Dinero capitalInicial)
    {
        if (capitalInicial.Monto <= 0) throw new DomainException("El capital debe ser mayor a cero.");
        var prestamo = new Prestamo(id, beneficiarioId, origen, condiciones, capitalInicial, new EstadoPendiente());
        prestamo.AddDomainEvent(new PrestamoCreado(prestamo.Id, beneficiarioId, capitalInicial));
        return prestamo;
    }

    // 3. RECONSTITUCIÓN DE DOMINIO (Exclusivo para la ACL Mapper desde la Base de Datos)
    public static Prestamo Reconstruir(PrestamoId id, BeneficiarioId beneficiarioId, OrigenApertura origen,
        CondicionesOriginales condiciones, Dinero saldoPendiente, EstadoPrestamo estado)
    {
```

10. Domain State Machine Pattern en C# 14

Eliminando `switch/if` Spaghetti con Records Polimórficos

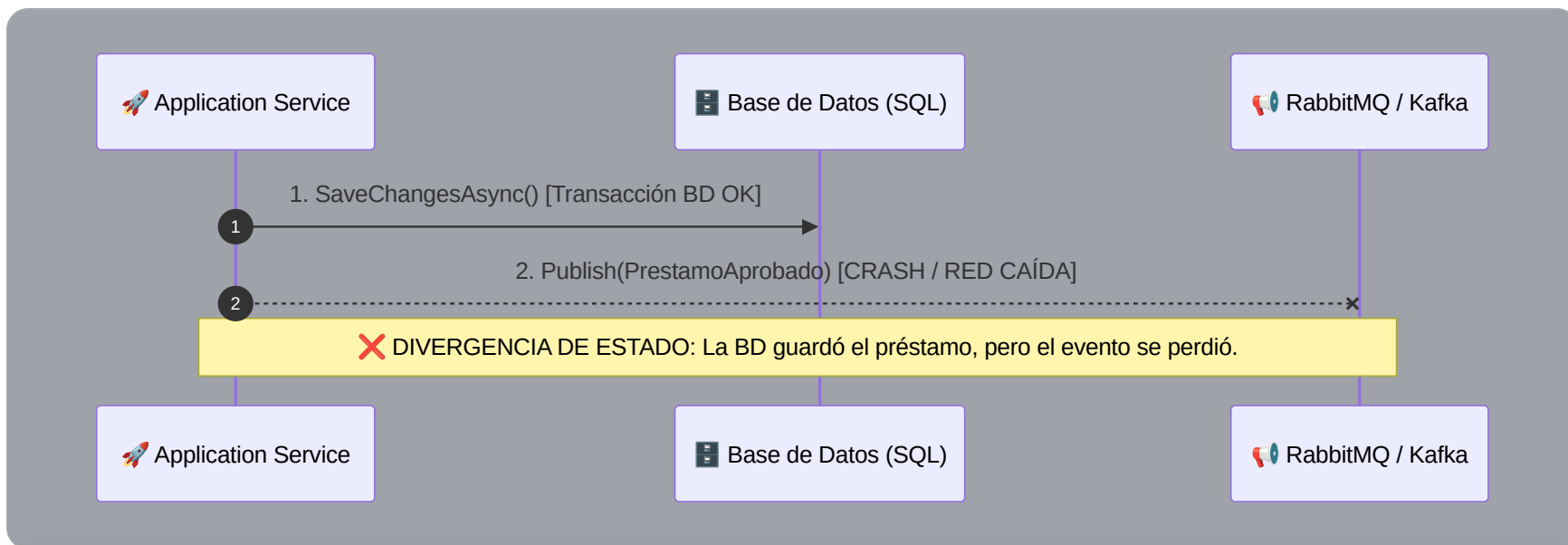
```
// Jerarquía polimórfica inmutable
public abstract record EstadoPrestamo(string Nombre)
{
    public abstract EstadoPrestamo TransicionarA(EstadoPrestamo nuevoEstado);
}

public record EstadoPendiente() : EstadoPrestamo("Pendiente")
{
    public override EstadoPrestamo TransicionarA(EstadoPrestamo nuevo) =>
        nuevo is EstadoAprobado or EstadoCancelado
            ? nuevo
            : throw new DomainException($"Transición inválida de Pendiente a {nuevo.Nombre}");
}

public record EstadoAprobado() : EstadoPrestamo("Aprobado")
{
    public override EstadoPrestamo TransicionarA(EstadoPrestamo nuevo) =>
        nuevo is EstadoLiquidado
            ? nuevo
            : throw new DomainException($"Un préstamo Aprobado solo puede pasar a Liquidado.");
}
```

II. El Problema del Dual-Write & EDA

¿Por qué falla guardar en BD y publicar en Broker en la misma petición?

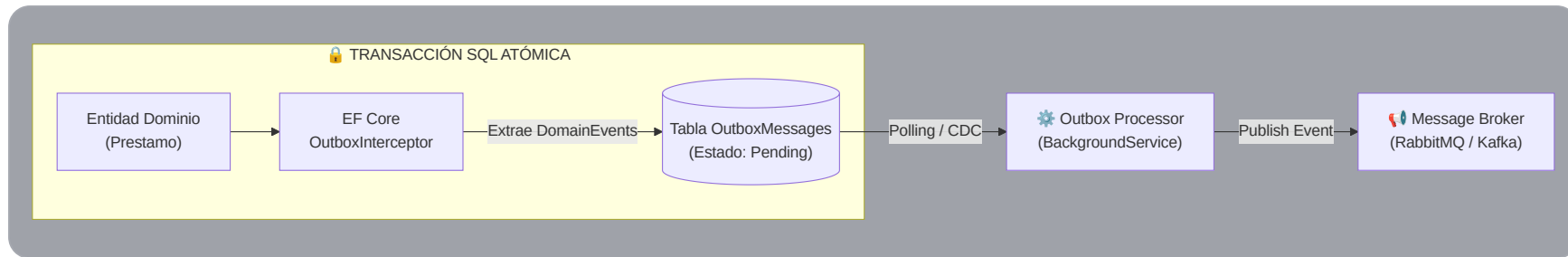


**DIVERGENCIA
CRÍTICA**

Publicar directamente al Message Broker tras guardar en BD no garantiza atomicidad. Se requiere el **OutboxPattern** para garantizar consistencia eventual.

12. Arquitectura del Outbox Pattern

Persistencia Atómica en una Única Transacción SQL



Los eventos se escriben en la tabla `OutboxMessages` dentro del mismo commit de la base de datos SQL.

13. Interceptor de Outbox en C# 14

OutboxDomainEventsInterceptor.cs

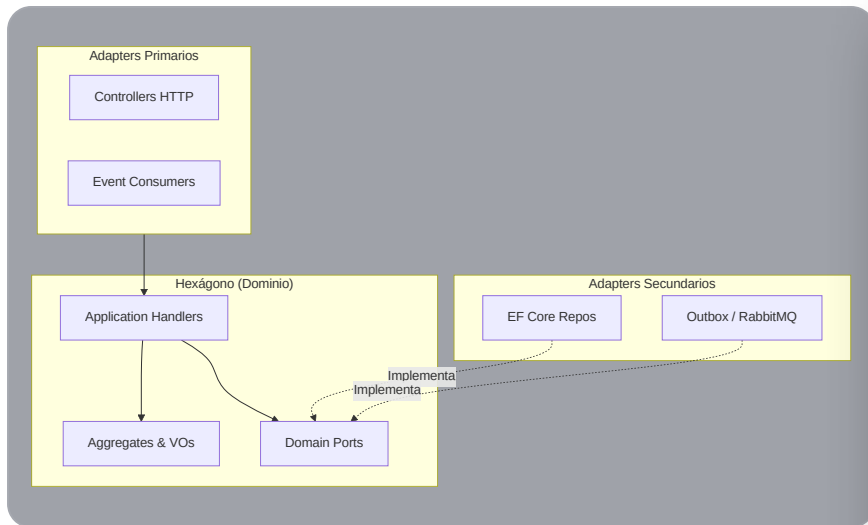
```
public class OutboxDomainEventsInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct = default)
    {
        var dbContext = eventData.Context;
        if (dbContext is null) return base.SavingChangesAsync(eventData, result, ct);

        // 1. Extraer Agregados con eventos de dominio pendientes
        var domainEvents = dbContext.ChangeTracker.Entries<IAggregateRoot>()
            .Select(e => e.Entity)
            .Where(e => e.DomainEvents.Any())
            .SelectMany(e => e.PopDomainEvents())
            .ToList();

        // 2. Convertir eventos en registros inmutables de Outbox
        var outboxMessages = domainEvents.Select(evt => new OutboxMessage(
            Guid.NewGuid(),
            evt.GetType().Name,
            JsonSerializer.Serialize(evt, evt.GetType()),
            DateTime.UtcNow
        ));
    }
}
```

14. Arquitectura Hexagonal

Puertos & Adaptadores en C# .NET 10



Principios de Inversión (DIP)

1. Puerto de Dominio:

`IPrestamoRepository : IRepository<Prestamo, PrestamoId>` vive en `Core.Prestamos.Domain.Ports`.

2. Adaptador Secundario:

`PrestamoRepository` implementa el puerto heredando de `GenericRepository` en `Persistence.Repository`.

3. Barrera ACL Integrada:

El repositorio invoca de forma transparente a `IMapper<Prestamo, PrestamoDbEntity>`.

15. CQRS Asimétrico & Result Pattern (Response)

Flujo de Control Funcional sin Excepciones Costosas

```
// 1. Result Pattern (Respuestas explícitas tipadas en lugar de throw new Exception)
public class Response
{
    public static Response<T> NotFound<T>(string error) => new(default, false, error);
    public static Response<Unit> NoContent(string message) => new(Unit.Value, true, message);
}

// 2. Command Handler Real (RegistrarPagoCommandHandler en Core.Prestamos)
public class RegistrarPagoCommandHandler(
    IPrestamoRepository prestamoRepository,
    IEstadoCuentaRepository estadoCuentaRepository,
    IUnitOfWork unitOfWork
) : ICommandHandler<RegistrarPagoCommand, Unit>
{
    public async Task<Response<Unit>> Handle(RegistrarPagoCommand request, CancellationToken cancellationToken)
    {
        var id = new PrestamoId(request.PrestamoId);
        var prestamo = await prestamoRepository.GetByIdAsync(id, cancellationToken);
        if (prestamo is null)
            return Response.NotFound<Unit>($"No se encontró el préstamo con ID '{request.PrestamoId}'.");

        var estadoCuenta = await estadoCuentaRepository.GetByIdAsync(id, cancellationToken);
        if (estadoCuenta is null)
            return Response.NotFound<Unit>($"No se encontró el estado de cuenta con ID '{request.EstadoCuentaId}'.");
        if (prestamo.Estado != PrestamoEstado.Pagado)
            return Response.BadRequest<Unit>("El préstamo no está en estado pagado.");
        if (estadoCuenta.Estado != EstadoCuentaEstado.Pagado)
            return Response.BadRequest<Unit>("El estado de cuenta no está en estado pagado.");
        unitOfWork.RegistrarPago(request, cancellationToken);
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return Response.NoContent("Pago registrado exitosamente.");
    }
}
```

16. Event Subscribers & Límite Transaccional

Suscriptores Delgados (*Slim Subscribers*): Evento → Mapeo a Comando → Mediador

Regla de Oro: Un Suscriptor de Eventos **NUNCA** inyecta `IRepository` , **ni** `DbContext` , **ni** `IUnitOfWork` . Modificar múltiples agregados en la misma transacción viola la encapsulación. Se inyecta exclusivamente `IMediator` y se despacha un Comando.

```
// 1. Evento de Dominio (Expresividad en pasado sin el sufijo redundante 'Event')
public record AfiliadoDesvinculado(int AfiliadoId, DateTime FechaDesvinculacion) : DomainEvent;

// 2. Suscriptor Delgado (Decoupled Slim Subscriber - Cero acceso directo a BD)
public class AfiliadoDesvinculadoSubscriber(
    IMediator mediator // ✅ ÚNICA DEPENDENCIA PERMITIDA: Mantiene la consistencia eventual aislada
) : IDomainEventSubscriber<AfiliadoDesvinculado>
{
    public async Task Handle(AfiliadoDesvinculado domainEvent, CancellationToken cancellationToken)
    {
        // 1. Mapeo transparente desde el Evento hacia un Comando de Aplicación
        var command = new LiquidarPrestamosPorDesvinculacionCommand(domainEvent.AfiliadoId);

        // 2. Despachar el comando. El CommandHandler abrirá su propia transacción aislada.
        await mediator.Send(command, cancellationToken);
    }
}
```

17. Checklist Architectural Linters

Reglas de Oro en DDD Enterprise



DO'S

Prácticas Obligatorias

- ✓ Constructores privados (``NuevoPrestamo`` creación vs ``Reconstruir`` hidratación).
- ✓ Eventos de Dominio en pasado sin sufijo `'Event'` (``PagoAutorizado``, ``PrestamoCreado``).
- ✓ Suscriptores Delgados: Inyectan **únicamente** ``IMediator`` y despachan Comandos.
- ✓ Outbox Pattern para publicación atómica de Domain Events en BD.
- ✓ Capa Anticorrupción (ACL/ ``IMapper``) para aislar tablas relacionales de BD.



DON'TS

Smells Prohibidos

- X Inyectar ``IRepository``, ``DbContext`` o ``IUnitOfWork`` dentro de un Event Subscriber.
- X Nombrar clases con el sufijo redundante ``Event`` (ej. ``PagoAutorizadoEvent``).
- X Entidades anémicas con setters públicos (``public set;``).
- X Lanzar excepciones para validaciones (``Response.UnprocessableEntity``).
- X ``switch/if`` spaghetti para manejo de estados (usar State Pattern).



Conclusión & Próximos Pasos

✓ Resumen Arquitectónico

- **1. Dominio Rico Primero:** Agregados protegidos con constructor privado, VOs con algoritmos y State Machine.
- **2. Bounded Contexts Aislados:** Usar ACL (`IMapper`) y `Reconstruir` para aislar datos.
- **3. Atomicidad Atómica:** `OutboxDomainEventsInterceptor` en tu DbContext SQL.
- **4. Hexágono & Suscriptores Delgados:** Puertos de Dominio y Suscriptores con `IMediator` (Agregado por Transacción).

¿Preguntas o Debate Técnico?



Ejecución local con Bun: `bun run dev`